

RELAZIONE TECNICA

Corso: Programmazione Web e Mobile

Anno Accademico: 2024/2025

Candidato: Damiano Ghibaudo

Data: 10 ottobre 2025

Indice

- 1. INTRODUZIONE E OBIETTIVI
 - 1.1 Panoramica del Progetto
 - 1.2 Vincoli Tecnici e Soluzioni Adottate
 - 1.3 Valore Aggiunto Implementato
- 2. ARCHITETTURA E DESIGN
 - 2.1 Struttura Modulare a Tre Layer
 - 2.2 Design Patterns
 - 2.3 Organizzazione File System
- 3. IMPLEMENTAZIONE
 - 3.1 Business Logic
 - 3.2 Gestione Ricette e API Integration
 - 3.3 Interfaccia Utente e UX
- 4. TEST e COMPATIBILITA'
 - 4.1 Requisiti di Compatibilità
 - 4.2 Test Funzionali
 - 4.3 Test Responsive Design
- 5. CONCLUSIONI
 - 5.1 Analisi Decisioni Architetture
 - 5.2 Conformità Requisiti
 - 5.3 Valutazione Scelte Tecnologiche
 - 5.4 Strategia Progettuale
 - 5.5 Implicazioni per Ambiente di Produzione
 - 5.6 Impatto sul Prodotto Finale

1. INTRODUZIONE E OBIETTIVI

1.1 Panoramica del Progetto

PGRC (Piattaforma per la Gestione di Ricette di Cucina) è un'applicazione web frontend-only sviluppata per soddisfare quattro macro-scenari funzionali:

1. **Gestione Profilo Utente:** Registrazione, autenticazione, modifica dati e cancellazione account
2. **Ricerca Ricette:** Integrazione con API TheMealDB per ricerca testuale e per categoria
3. **Ricettario Personale:** Sistema preferiti con note private personalizzabili
4. **Sistema Recensioni:** Rating duale (gusto/difficoltà) con aggregazione community

1.2 Vincoli Tecnici e Soluzioni Adottate

Il progetto rispetta i vincoli di consegna implementando le seguenti soluzioni e tecnologie:

- **Frontend Only:** HTML5, CSS3, JavaScript ES6+ senza dipendenze backend
- **Persistenza Locale:** Web Storage API (localStorage/sessionStorage) con architettura cache-first
- **Separazione Concerns:** Architettura modulare a tre layer (Presentation, Service, Data)
- **Responsive Design:** Mobile-first approach con CSS Grid e classi Bootstrap

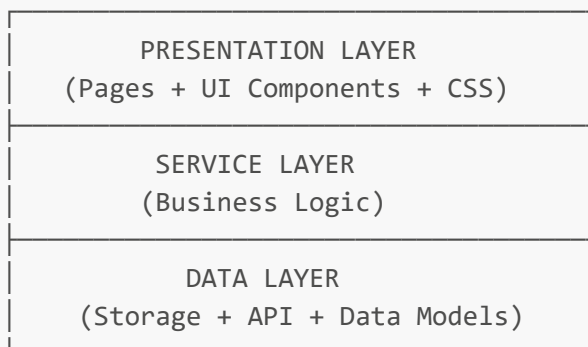
1.3 Valore Aggiunto Implementato

Oltre ai requisiti base, l'applicazione introduce:

- Session Service come orchestratore centrale con facade pattern
- Sistema di validazione dual-layer (UX + business logic) per dati utente
- Error handling tipizzato con classi custom
- UI dinamica con event delegation pattern

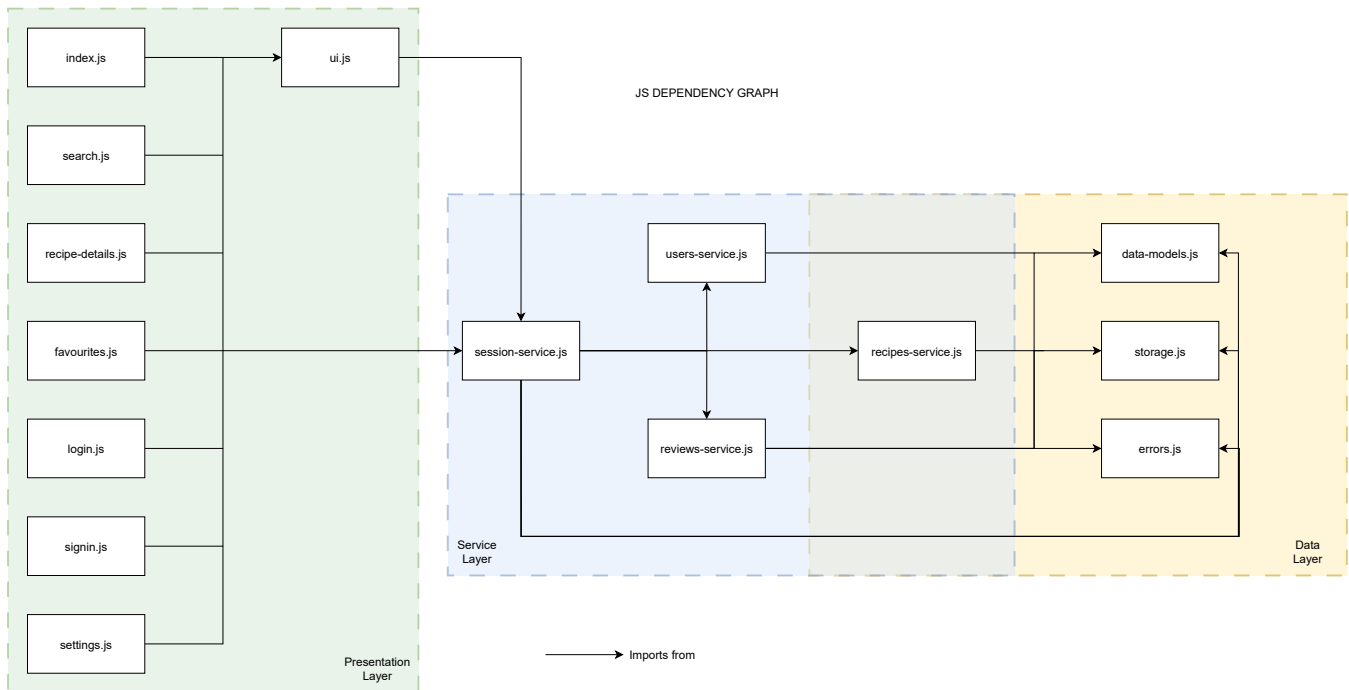
2. ARCHITETTURA E DESIGN

2.1 Struttura Modulare a Tre Layer



2.1.1 Presentation Layer

Le pagine HTML implementano struttura con separazione netta tra contenuto e presentazione. I componenti UI in ui.js gestiscono rendering dinamico e interazioni, mentre i page scripts in js/pages/ orchestrano gli eventi e la logica specifica di ogni pagina.



2.1.2 Service Layer

Coordina la business logic attraverso quattro moduli specializzati:

- **session-service.js:** Orchestratore centrale che espone API unificate tramite namespace per gestione stati e operazioni business
- **users-service.js:** Gestione CRUD utenti, autenticazione sicura con hashing SHA-256, validazione duplicati e operazioni profilo
- **recipes-service.js:** Integrazione API TheMealDB con strategia cache-first, normalizzazione dati e algoritmi di ricerca
- **reviews-service.js:** Gestione CRUD recensioni, sistema di rating duale e aggregazione statistiche

2.1.3 Data Layer

Gestisce persistenza tramite storage.js (astrazione localStorage/sessionStorage), integrazione con API esterna tramite recipes-service.js e modelli dati con data-models.js.

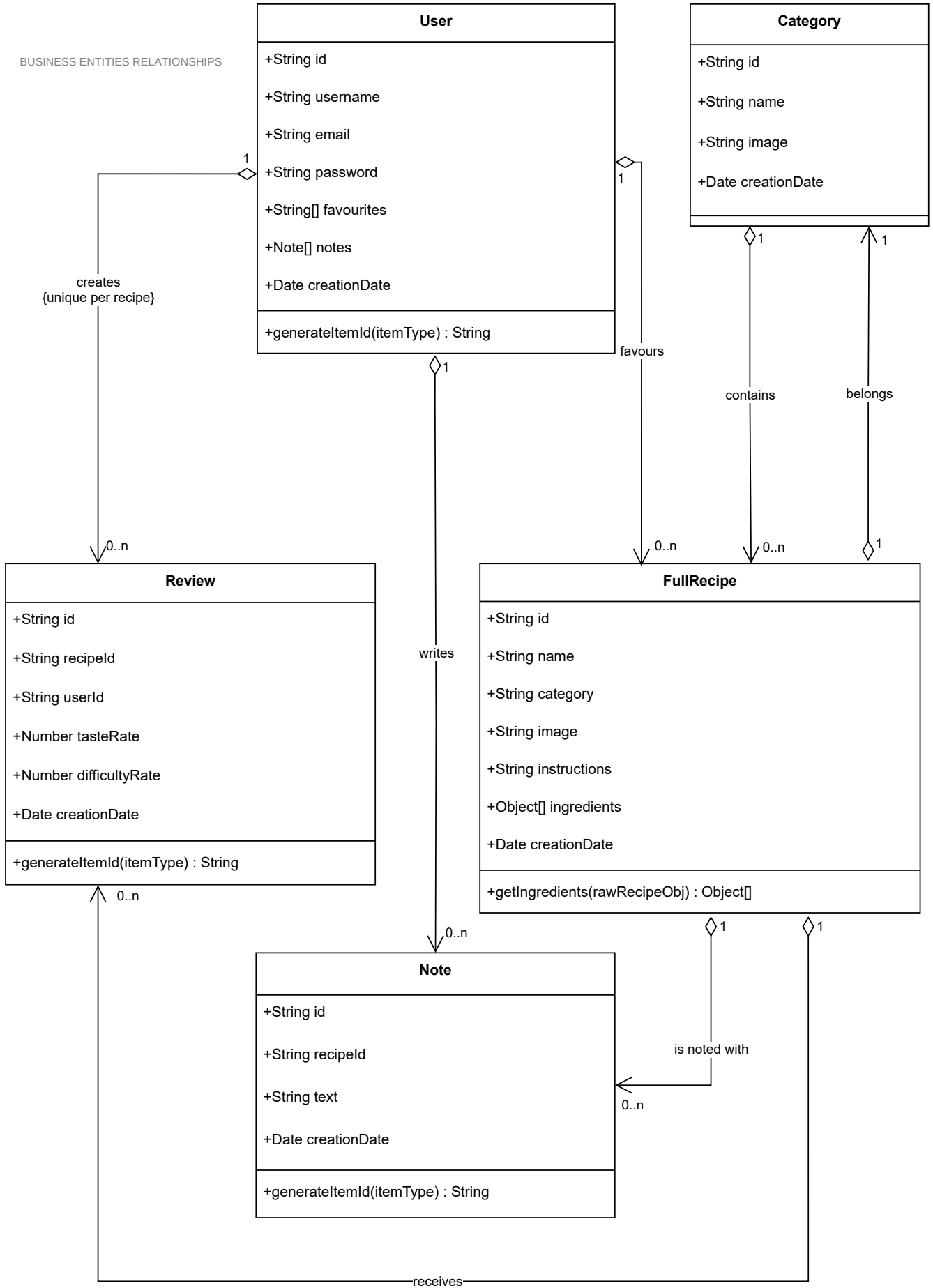
2.2 Design Patterns

2.2.1 Data Models

Il sistema utilizza classi ES6 per la creazione consistente di entità business:

- **User class:** Genera istanze utente con ID univoci, dati personali e array inizializzati per preferiti/notes
- **FullRecipe e Category classes:** Normalizzano risorse da API TheMealDB in formato interno consistente
- **Review class:** Crea recensioni con validazione parametri e timestamp automatici
- **ID Generation:** Sistema centralizzato per generazione identificatori univoci basati su timestamp e randomizzazione

BUSINESS ENTITIES RELATIONSHIPS



2.2.2 Error Handling

Sistema di gestione errori tipizzato con classi custom per scenari specifici:

- **NotFound (404):** Entità non trovate con dettagli campo/valore per debugging
- **Duplicated (409):** Violazioni unicità con identificazione campo duplicato
- **InvalidFormat (422):** Errori validazione formato con messaggio specifico

Ogni classe fornisce un codice errore compatibile con HTTP status codes e metadati per logging centralizzato.

2.2.3 Event Delegation

Il sistema implementa event delegation per gestire interazioni su contenuto generato dinamicamente.

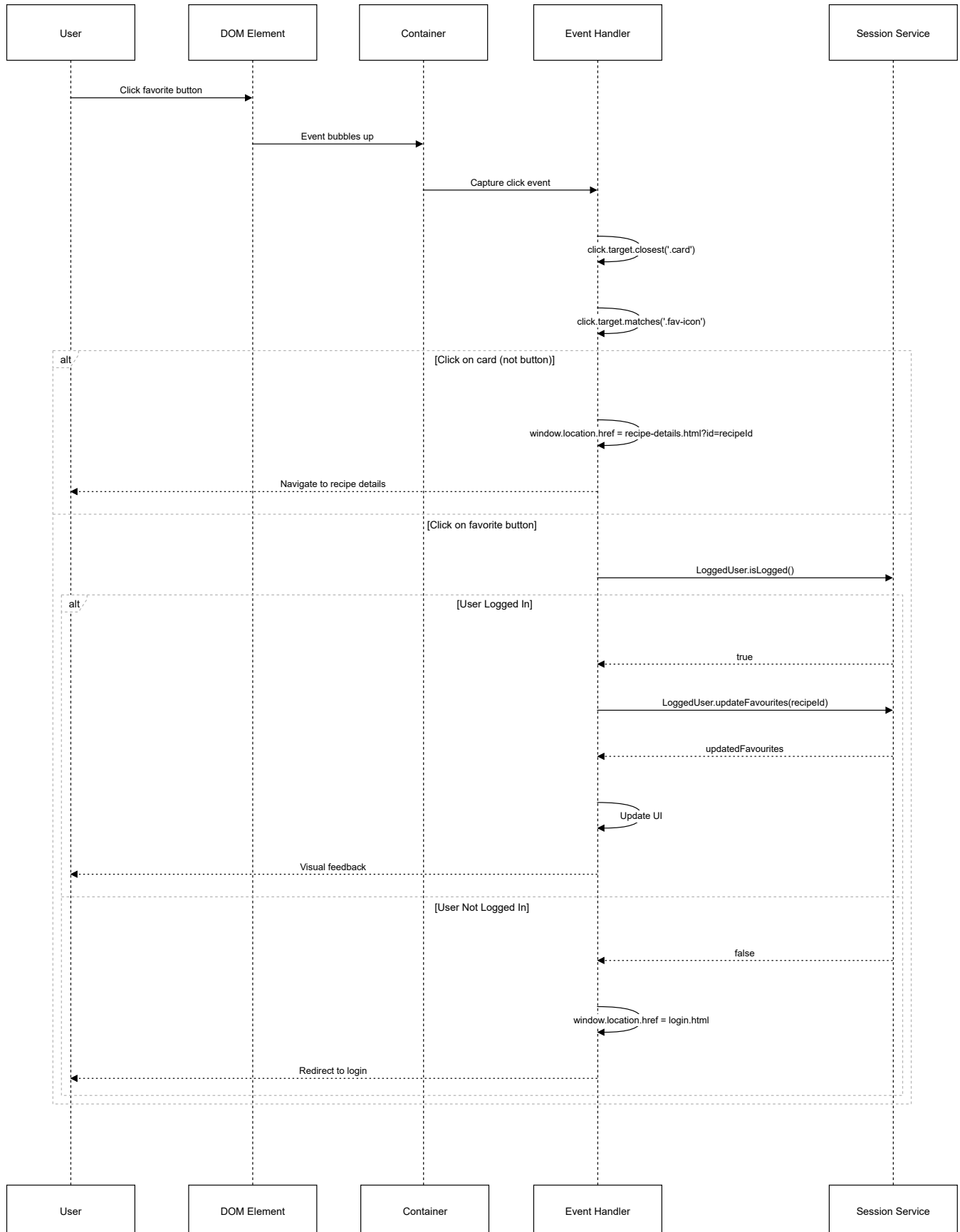
Strategia implementata:

- **Single Event Listener:** listener sul container padre gestisce eventi di tutti gli elementi figli
- **Target Detection:** Utilizzo di `closest()` e `matches()` per l'identificazione dell'elemento cliccato
- **Action Routing:** Logica condizionale per instradare verso azioni diverse (es. navigazione vs. toggle preferiti)
- **Authentication Guard:** Controllo stato utente per proteggere operazioni riservate

Vantaggi tecnici:

- **Performance:** Riduzione memoria con un solo listener invece di N listener per N elementi
- **Dinamicità:** Gestione automatica di elementi aggiunti/rimossi dinamicamente
- **Manutenibilità:** Centralizzazione logica eventi in un singolo punto

EVENT DELEGATION SEQUENCE



2.3 Organizzazione File System

```
PGRC-project/
├── index.html          # Homepage con carousel ricette popolari
├── style.css           # Stili CSS principali responsive
├── pages/              # Pagine HTML dell'applicazione
│   ├── favourites.html # Ricettario personale con tab
│   ├── login.html      # Autenticazione utente
│   ├── recipe-details.html # Dettaglio ricetta completo
│   ├── search.html     # Ricerca e risultati
│   ├── settings.html   # Gestione profilo utente
│   └── signin.html     # Registrazione nuovo utente
├── js/                # Moduli JavaScript
│   ├── core/          # Moduli fondamentali
│   │   ├── data-models.js # Factory per entità business
│   │   ├── errors.js     # Error classes tipizzate
│   │   └── storage.js    # Astrazione Web Storage unificata
│   ├── services/      # Business logic layer
│   │   ├── session-service.js # Orchestratore centrale con namespace
│   │   ├── users-service.js  # CRUD utenti + sicurezza + validazione
│   │   ├── recipes-service.js # API integration + cache TheMealDB
│   │   └── reviews-service.js # Sistema recensioni con aggregazione
│   ├── components/    # Componenti UI riutilizzabili
│   │   └── ui.js       # Factory card, rating display, popolamento
└── container
    ├── pages/          # Script specifici per pagina
    │   ├── index.js    # Homepage con carousel e categorie
    │   ├── login.js    # Autenticazione e gestione sessione
    │   ├── signin.js   # Registrazione con validazione real-time
    │   ├── search.js   # Ricerca ricette e gestione risultati
    │   ├── recipe-details.js # Dettaglio ricetta con recensioni e note
    │   ├── favourites.js # Ricettario personale con tab
    │   └── settings.js  # Modifica profilo e cancellazione account
    ├── assets/         # Risorse statiche
    ├── bootstrap/      # Framework CSS/JS
    └── utils/          # Utilità sviluppo
```

3. IMPLEMENTAZIONE

3.1 Business Logic

3.1.1 Entità e Regole di Business

Il progetto è strutturato attorno a quattro moduli di servizio principali che gestiscono le operazioni business: gestione sessione, gestione utenti, gestione ricette e categorie e gestione recensioni, con preferiti e note integrate come funzionalità del profilo utente.

Gestione Sessione session-service.js

- **Orchestrazione Centrale:** Coordina tutte le operazioni business attraverso namespace specializzati (NewUser, LoggedUser, Recipe, PreviewArray)
- **Stato Autenticazione:** Gestisce login/logout e persistenza ID utente tramite sessionStorage
- **Facade Pattern:** Espone API unificate per operazioni CRUD su utenti, ricette e recensioni

Gestione Utenti users-service.js

- **CRUD Completo:** Registrazione, autenticazione, modifica profilo e cancellazione account
- **Sicurezza:** Hashing SHA-256 password con Web Crypto API, validazione requisiti, validazione unicità email/username
- **Profilo Utente:** Gestione preferiti e note personali associate a ricette
- **Persistenza:** Salvataggio dati utenti in localStorage

Gestione Ricette recipes-service.js

- **Integrazione API:** Connessione con TheMealDB per contenuti ricette
- **Dati Locali:** Persistenza ricette in localStorage con refresh automatico
- **Ricerca Avanzata:** Algoritmo di scoring multi-termine per risultati rilevanti
- **Normalizzazione Dati:** Conversione formato API in modelli interni standardizzati
- **Categorie:** Classificazione ricette da TheMealDB con navigazione per filtri

Gestione Recensioni reviews-service.js

- **Rating Duale:** Valutazione gusto (1-5) e difficoltà (1-5) per ogni ricetta
- **Vincolo Unicità:** Un utente può recensire una ricetta una sola volta
- **Aggregazione Statistiche:** Calcolo medie community per ranking ricette
- **Integrità Referenziale:** Gestione trasferimento recensioni in caso di cancellazione utente

3.1.2 Flussi Operativi e Gestione Stati

L'applicazione gestisce due tipologie di utenti con flussi operativi distinti.

Utente Non Autenticato (Guest):

- Accesso alla homepage con ricette popolari
- Ricerca ricette per nome o categoria
- Visualizzazione dettagli ricetta (senza possibilità di interazione)
- Accesso limitato: no preferiti, no recensioni, no note

Utente Autenticato:

- Tutte le funzionalità guest più:
- Gestione preferiti (aggiunta/rimozione ricette)
- Sistema recensioni con rating duale gusto/difficoltà
- Note personali private per ogni ricetta
- Modifica profilo e gestione account

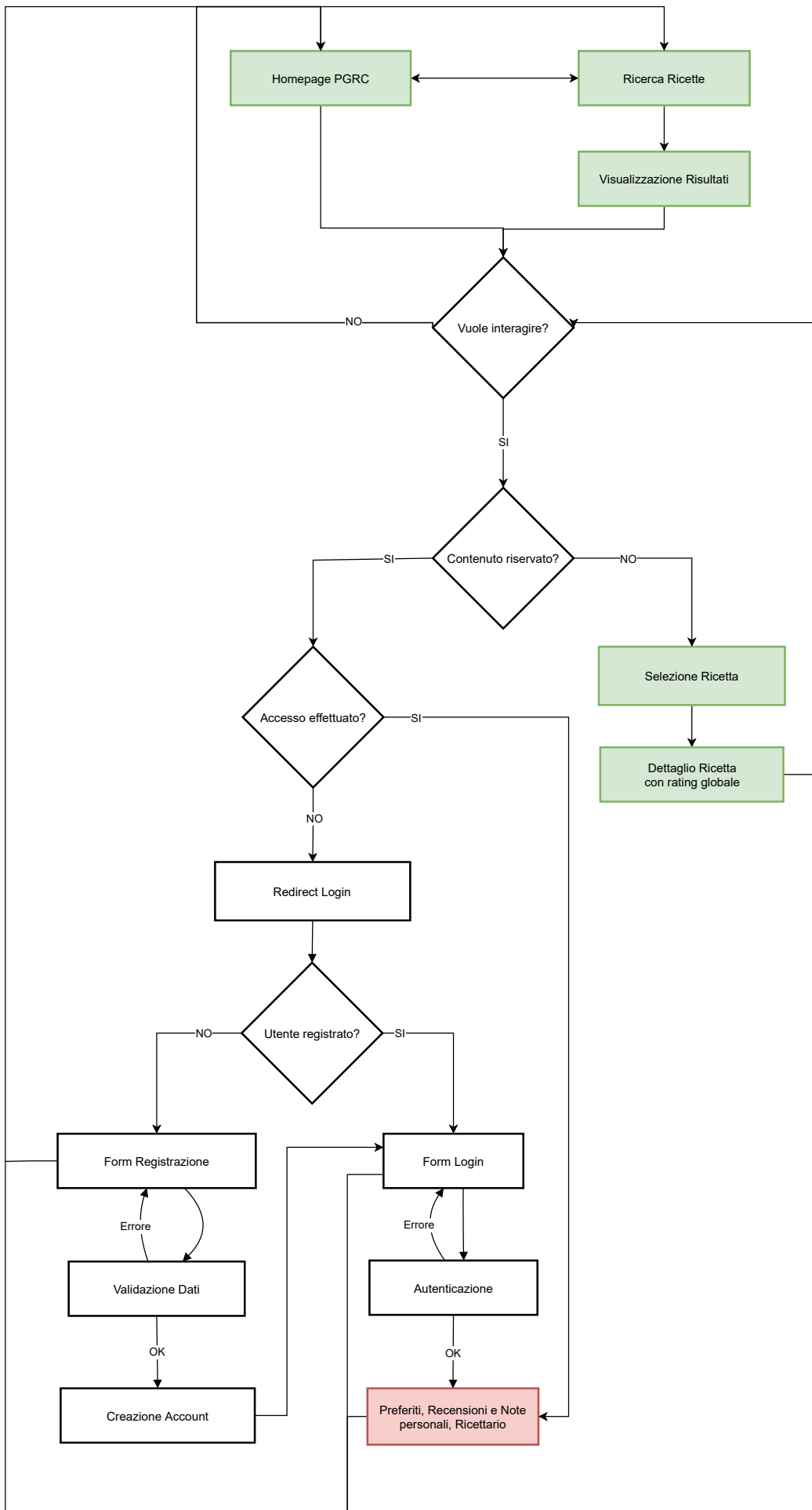
Processo di Registrazione e Login:

1. **Registrazione:** L'utente compila il form con username, email, password
2. **Validazione:** Il sistema verifica unicità email/username e requisiti password
3. **Salvataggio:** I dati vengono salvati in localStorage con password hashata
4. **Login:** Verifica credenziali e creazione sessione utente
5. **Stato Attivo:** L'ID utente viene memorizzato in sessionStorage per la durata della sessione

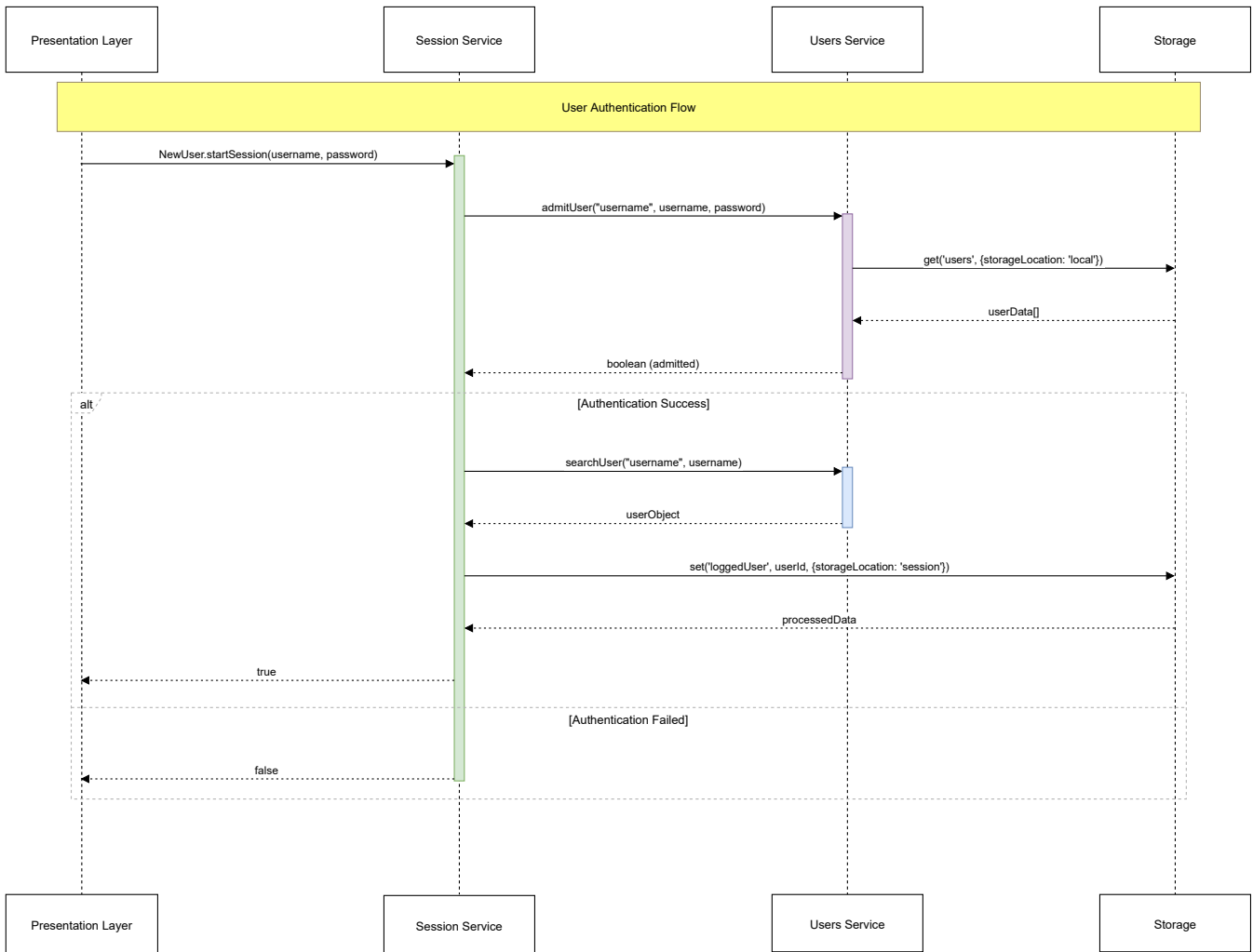
Persistenza Dati:

- **SessionStorage:** Mantiene l'ID dell'utente loggato fino alla chiusura della scheda corrente
- **LocalStorage:** Conserva permanentemente database utenti, preferiti, recensioni e note
- **Sincronizzazione UI:** L'interfaccia si aggiorna automaticamente in base allo stato di login

FLUSSO NAVIGAZIONE UTENTE



USER LOGIN SEQUENCE



3.1.3 Orchestrazione Centralizzata tramite Session Service

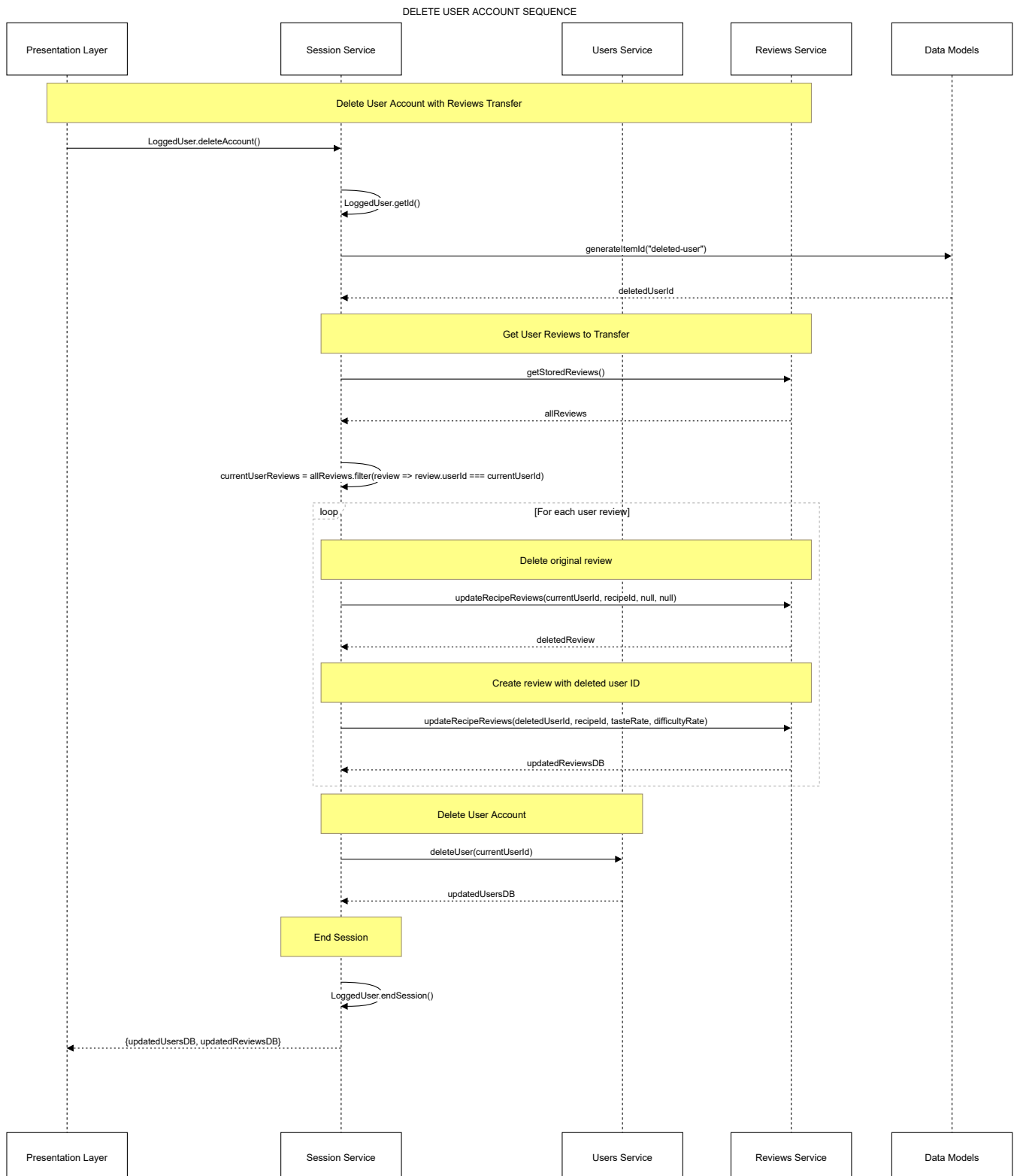
Il Session Service (`session-service.js`) implementa il pattern Facade per coordinare tutte le operazioni business attraverso namespace specializzati:

Namespace NewUser

Gestisce le operazioni disponibili per utenti non autenticati. Include funzionalità per avviare una sessione utente attraverso autenticazione con credenziali (username e password), registrare nuovi utenti nel database con validazione completa dei dati (username, email, password con conferma) ed eseguire controlli di validazione real-time sui campi di input per garantire conformità ai requisiti di formato e unicità prima dell'invio dei form.

Namespace LoggedUser

Interfaccia centralizzata per tutte le operazioni che richiedono autenticazione utente. Coordina l'accesso ai servizi di gestione utenti (`users-service.js`), ricette (`recipes-service.js`) e recensioni (`reviews-service.js`) orchestrando le chiamate ai moduli sottostanti. Include funzionalità per la modifica del profilo personale con controlli di unicità e integrità, gestione delle preferenze utente (aggiunta/rimozione preferiti) con aggiornamenti bidirezionali tra stato applicativo e persistenza locale, aggiunta recensioni e note private, gestione della sessione corrente attraverso meccanismi di logout sicuro ed eliminazione dell'account con cancellazione completa dei dati personali ed aggiornamento della proprietà delle recensioni associate.



Namespace PreviewArray:

Fornisce funzioni specializzate per il recupero e la preparazione di dati strutturati destinati al rendering dell'interfaccia utente, coordinando l'accesso ai servizi di ricette (`recipes-service.js`) e recensioni (`reviews-service.js`). Ogni funzione restituisce un oggetto uniforme con due campi: **type** (stringa che identifica il tipo di contenuto: `categories`, `meals`, `reviews`, `notes`) e **items** (array contenente i dati elaborati). Le funzionalità includono recupero di categorie disponibili, ricerca ricette per nome o categoria, estrazione di array specifici per ID o casuali, selezione di ricette recensite/preferite/note dell'utente autenticato e ordinamento delle ricette più popolari basato sul numero di recensioni.

Namespace Recipe:

Interfaccia centralizzata per interrogare lo stato delle ricette in relazione all'utente autenticato. Coordina l'accesso ai servizi specializzati di gestione ricette (recipes-service.js) e recensioni (reviews-service.js). Include funzionalità per l'accesso a informazioni contestuali come lo stato di preferenza, presenza di recensioni e note personali, valutazioni aggregate e operazioni CRUD per l'aggiornamento delle recensioni.

3.1.4 Sicurezza e Validazione

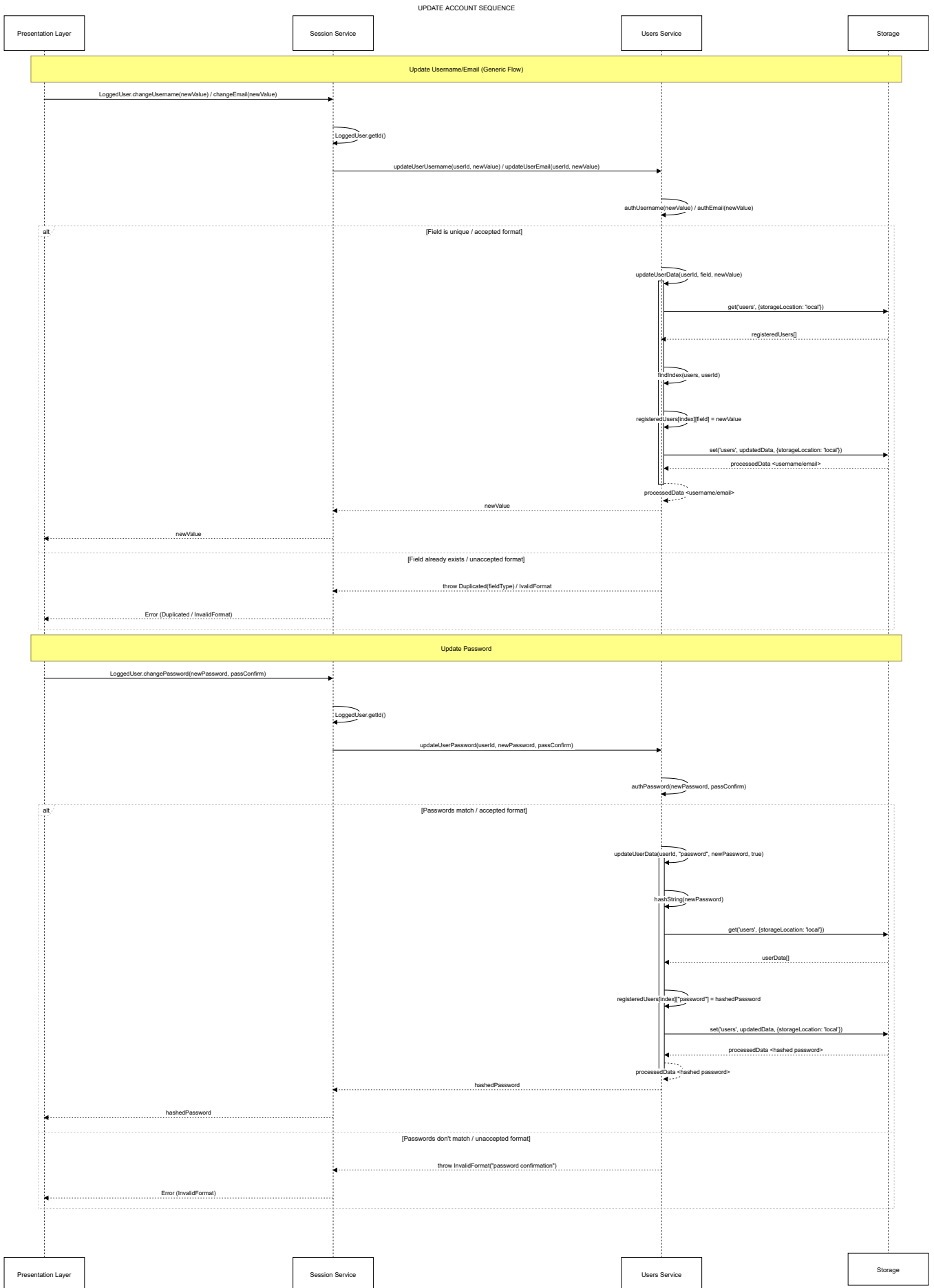
Il sistema implementa una strategia di validazione a due livelli che garantisce sia l'esperienza utente che l'integrità dei dati.

Layer di Presentazione:

- **Validazione Tempo Reale:** Durante la digitazione nei campi form, JavaScript esegue controlli immediati di formato tramite la funzione `formatInputField()` che orchestra validazione UI e business logic
- **Feedback Visivo Dinamico:** Applicazione delle classi Bootstrap-valid/is-invalid per evidenziare lo stato di validazione dei campi (formato e unicità) con cambio colore bordi e visualizzazione di indicazioni testuali
- **Gestione Stati Pulsante Submit:** Il pulsante di invio rimane disabilitato fino al completamento positivo di tutti i controlli di validazione richiesti
- **Disabilitazione Condizionale Campi:** Campi dipendenti (es. conferma password) vengono abilitati solo dopo validazione positiva del campo di riferimento

Layer Business effettua ulteriore validazione tramite controlli rigorosi prima della persistenza dei dati:

- **Email e Username:** controllo formato (regex per email) e unicità del valore all'interno del set di utenti registrati
- **Password:** controllo requisiti di complessità (8 caratteri, lettere maiuscole/minuscole, numeri), hashing (SHA256) prima di salvataggio in fase di registrazione e confronto in caso di login.



3.1.5 Controllo del flusso e gestione degli errori

Il sistema adotta un approccio basato su **gestione delle eccezioni come controllo del flusso** invece di utilizzare logica condizionale tradizionale per le decisioni business. Questa scelta strutturale influenza profondamente l'organizzazione del codice e la gestione degli stati dell'applicazione.

- **Business Modules:** Operazioni di validazione e ricerca lanciano eccezioni tipizzate in caso di valori duplicati, risorse non trovate o formati non accettati (oltre errori di sistema ed eccezioni critiche)
- **Session service:** Tutti i namespace implementano pattern catch and rethrow con propagazione trasparente
- **Presentation Layer:** Try-catch localizzati con type checking tramite codice errore e risoluzione tramite logica decisionale, graceful degradation, feedback UI ed eventuale log con stack-trace completo (per eccezioni di sistema e/o critiche)

Caratteristiche del Pattern:

- **Operazioni Atomiche e validazione fail-fast:** Le operazioni vengono interrotte immediatamente, con fallimento completo, alla prima validazione fallita
- **Linearità del Codice:** Eliminazione di if/else nidificati
- **Code-Based Error Handling:** Utilizzo di controlli `error.code` per gestione specifica per ogni classe di errore custom (HTTP compliant)

Vantaggi Architetturali:

- **Consistenza:** Stesso pattern exception-first utilizzato uniformemente in tutto il codebase
- **Manutenibilità:** Nessuna ambiguità nella gestione delle eccezioni (meccanismo unico)
- **Chiarezza e Leggibilità:** Separazione netta tra flusso normale e situazioni eccezionali

3.1.6 Storage Strategy Unificata

Il sistema di storage unificato (storage.js) astrae localStorage/sessionStorage attraverso il namespace StorageOperations:

Operazioni Core:

- `get(storageKey, options):` Recupero dati con deserializzazione automatica per tipo
- `set(storageKey, data, options):` Salvataggio con serializzazione automatica JSON

Gestione Tipi e Opzioni:

- Parsing automatico JSON per array con fallback array vuoto
- Gestione string con fallback stringa vuota
- Selezione storage engine: `{storageLocation: "local"|"session", dataType: "array"|"string"}`

3.2 Gestione Ricette e API Integration

3.2.1 Persistenza Locale dei Dati API

L'applicazione implementa un sistema di persistenza locale per i dati provenienti dall'API esterna, ottimizzando le prestazioni attraverso la riduzione delle chiamate di rete e garantendo la disponibilità offline dei contenuti

Meccanismo di Aggiornamento Temporale: I dati API vengono memorizzati in localStorage con un timestamp di creazione. Ad ogni richiesta di accesso ai dati, il sistema confronta il timestamp memorizzato con la data corrente. Se i dati sono stati creati in una giornata precedente, viene automaticamente avviata una procedura di recupero dei dati aggiornati dall'API esterna, che vengono poi memorizzati localmente sostituendo quelli obsoleti.

Ottimizzazione delle Performance: Questa strategia garantisce che le successive richieste dello stesso utente nella stessa giornata utilizzino i dati locali, eliminando latenze di rete e migliorando significativamente la responsività dell'interfaccia utente.

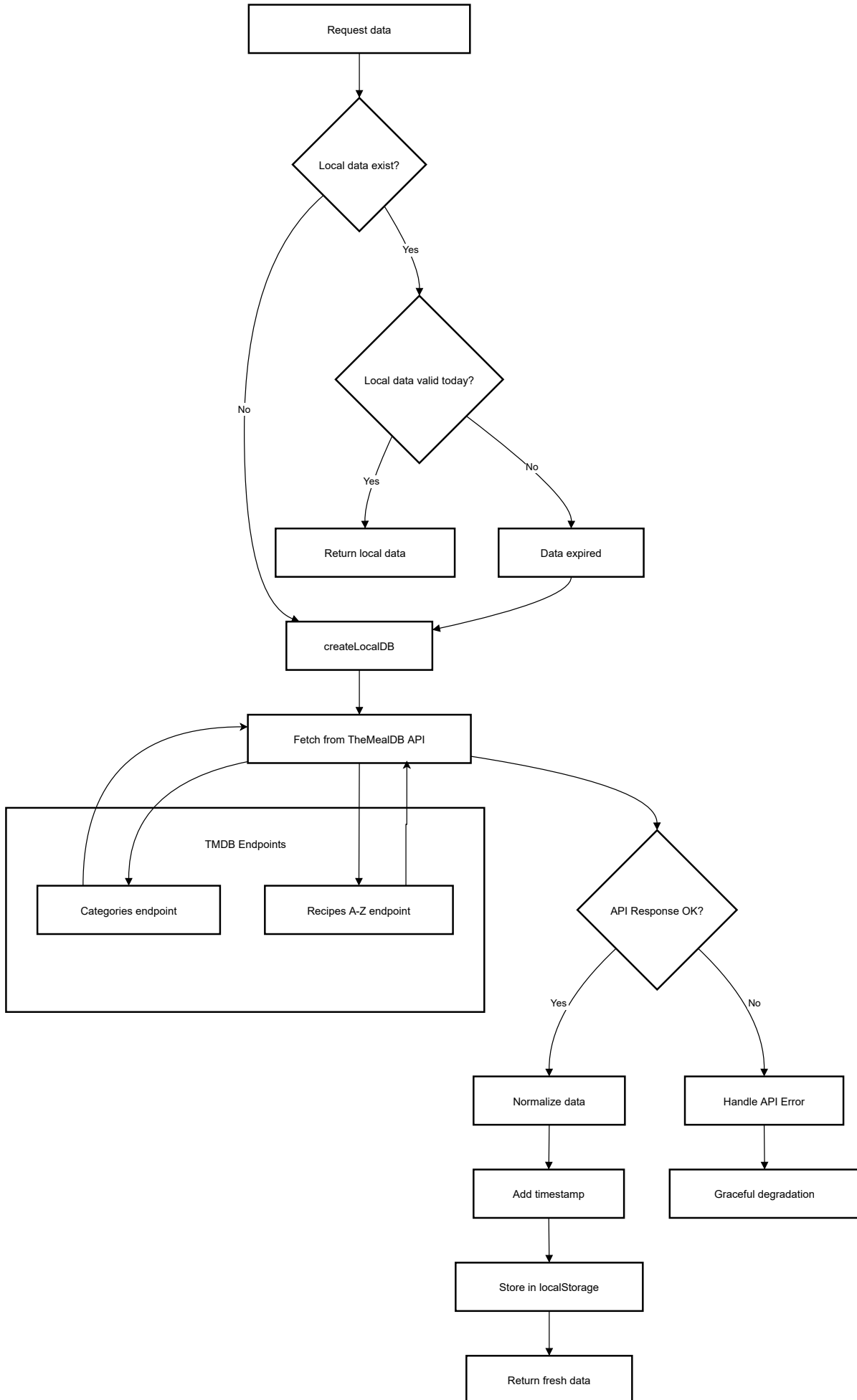
3.2.2 Integrazione con TheMealDB API

L'applicazione si interfaccia con con l'API TheMealDB attraverso due endpoint:

- **Lista Categorie:** <https://www.themealdb.com/api/json/v1/1/categories.php>
- **Ricerca per Lettera:** <https://www.themealdb.com/api/json/v1/1/search.php?f={letter}>

Caricamento Completo del Database: Al fine di superare le limitazioni dell'API TheMealDB (assenza di un endpoint "get all recipes"), il sistema implementa una strategia di caricamento alfabetico che itera attraverso tutte le lettere dell'alfabeto (A-Z) per scaricare l'intero database delle ricette.

RECIPES DATA PERSISTENCE FLOW



3.2.3 Algoritmo di Ricerca

La funzionalità di ricerca per nome ricetta implementa un algoritmo di scoring che elabora query multi-termini attraverso una strategia di matching ponderato. L'algoritmo opera secondo i seguenti passi:

- **Normalizzazione Input:** Sia la query di ricerca che i nomi delle ricette vengono convertiti in minuscolo e privati di spazi iniziali/finali per garantire confronti case-insensitive e robusti.
- **Tokenizzazione:** La query normalizzata viene suddivisa in termini di ricerca utilizzando espressioni regolari che riconoscono sequenze di spazi come delimitatori.
- **Scoring Iterativo per Ricetta:** Per ciascuna ricetta nel database locale, l'algoritmo:
 - Suddivide il nome della ricetta in termini individuali
 - Inizializza un punteggio cumulativo e un contatore di termini matched
 - Per ogni termine di ricerca, itera attraverso i termini del nome applicando regole di scoring
- **Regole di Scoring Individuali:**
 - Match Esatto (20 punti): Assegnato quando un termine del nome corrisponde esattamente al termine di ricerca
 - Match Parziale (10 punti): Assegnato quando un termine del nome inizia con il termine di ricerca
 - Penalizzazione Posizionale: I punteggi vengono divisi per un divisore incrementale che penalizza termini meno prominenti nel nome (primi termini ottengono punteggi più elevati)
 - Prevenzione Conteggi Multipli: Una volta trovato un match per un termine di ricerca, vengono evitati ulteriori conteggi per lo stesso termine
- **Bonus per Completezza:** Se tutti i termini della query trovano corrispondenza nella ricetta, il punteggio totale viene raddoppiato per premiare match completi.
- **Ordinamento e Filtraggio:** Le ricette con punteggio positivo vengono ordinate per rilevanza decrescente, garantendo che i risultati più pertinenti appaiano primi nell'interfaccia utente.

3.2.4 Modello di Dati Normalizzati

La classe FullRecipe nel modulo data-models.js effettua la normalizzazione dei dati provenienti dall'API TheMealDB:

- **Mapping dei Campi:** Conversione automatica dal formato API alla struttura interna standardizzata
- **Parsing degli Ingredienti:** Metodo getIngredients(rawRecipeObj) per l'estrazione degli ingredienti con relative misure dai campi numerati (1-20)
- **Valori di Fallback:** Gestione dei campi mancanti mediante valori predefiniti e immagine placeholder
- **Creazione Timestamp:** Aggiunta automatica del campo `creationDate` per la gestione della cache

3.3 Interfaccia Utente e UX

3.3.1 Responsive Design Mobile-First

L'approccio mobile-first è implementato tramite CSS Grid e media queries con breakpoint ottimizzati:

- **Mobile (320px+):** Layout single-column con card ricetta verticali
- **Tablet (560px+):** Grid a 2 colonne con card ricetta orizzontali
- **Desktop (720px+):** Grid ottimizzata multicolonna

La navbar utilizza Bootstrap 5 con design fixed-top e dropdown menu per la navigazione, mentre i container di ricette implementano CSS Grid responsive con `auto-fill` e `minmax()` per adattamento automatico.

3.3.2 Architettura Componenti UI

Il sistema di rendering (ui.js) implementa un'architettura component-based con factory pattern per la generazione dinamica di elementi DOM:

Factory Pattern per Elementi DOM: Implementato nelle funzioni private che generano elementi DOM standardizzati come `createPreviewCard()`. Ogni elemento include data attributes per event delegation e supporta inserimento condizionale di contenuto body specifico.

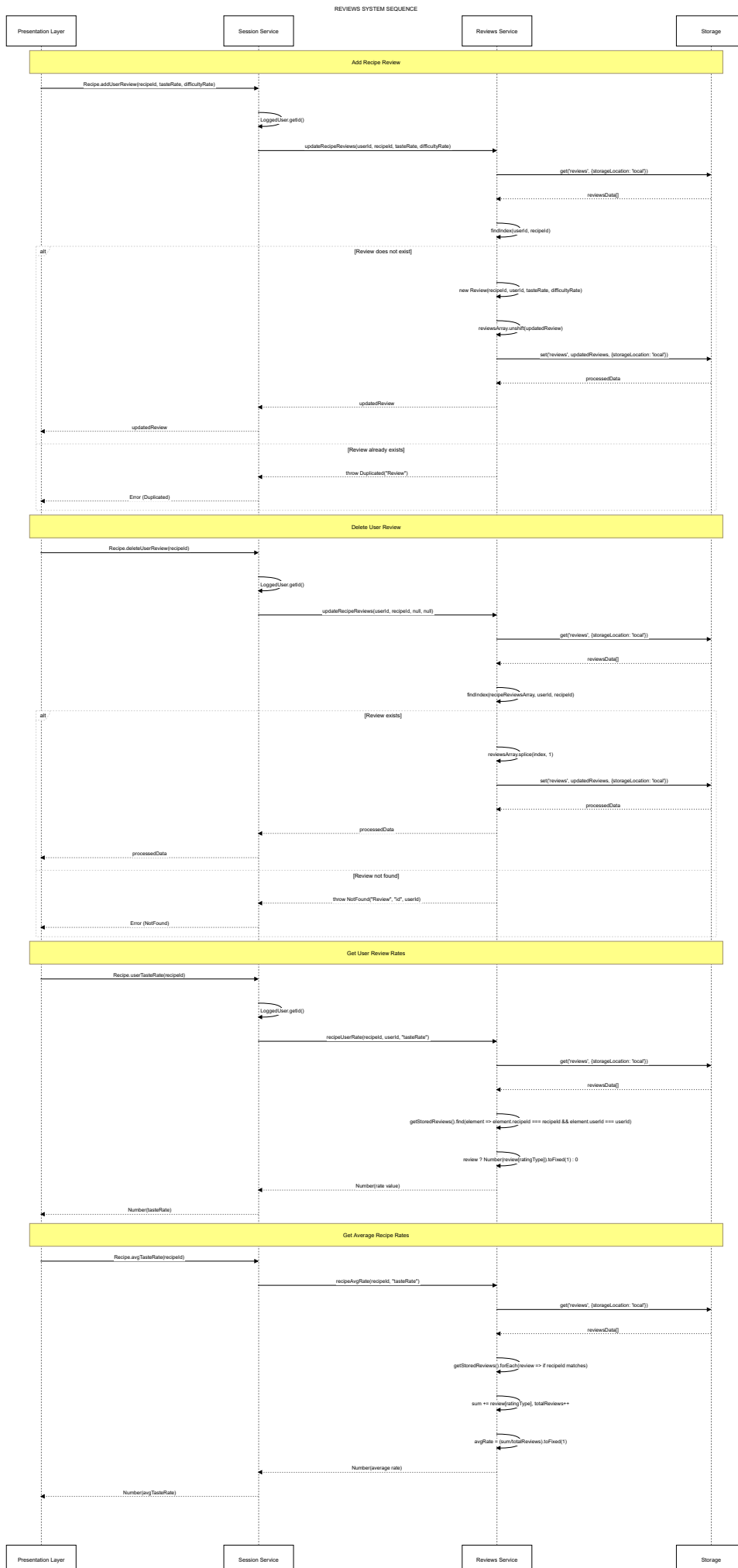
Strategy Pattern e Type-Based Rendering: Il sistema implementa uno strategy pattern per la gestione dinamica del rendering dei contenuti nelle card delle ricette, selezionando strategie di rendering specifiche basate sul tipo di dati e sul contesto utente. La funzione `populatePreviewContainer` applica direttamente la strategia di rendering corrispondente dal namespace `CardDisplayStrategy` in base al valore del campo `type` dell'oggetto ricevuto, eliminando la necessità di logica condizionale esterna e consentendo di gestire in modo modulare ed estensibile la visualizzazione di nuovi tipi di contenuto nell'interfaccia utente.

3.3.3 Gestione interazioni

Nelle funzioni di aggiornamento (`updateRecipeReviews()`, `updateFavourites()`, `updateNotes()`) è implementato un pattern per la gestione unificata di operazioni CRUD su entità relazionali utente-contenuto (recensioni, preferiti, note personali), garantendo consistenza e atomicità attraverso meccanismi di toggle intelligente. Questo pattern unifica creazione e rimozione attraverso un'unica funzione che determina l'operazione basandosi sulla presenza/assenza di parametri specifici:

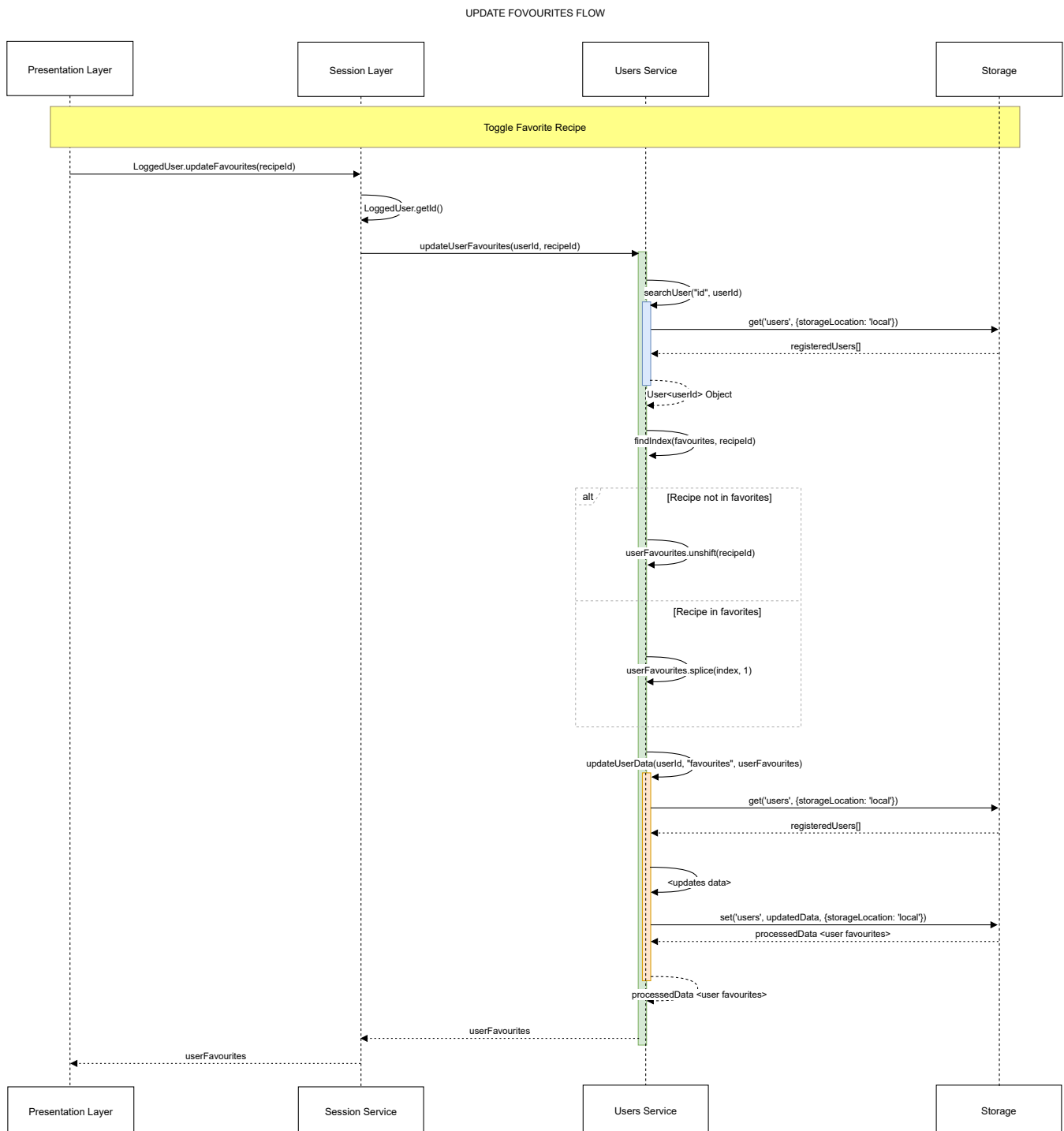
Toggle per Recensioni:

- **ADD:** tutti i parametri forniti; crea nuova recensione con validazione unicità relazionale
- **DELETE:** solo identificatori essenziali forniti (`userId` + `recipeId`); rimuove recensione esistente con validazione presenza



Toggle per Preferiti:

- ADD: Se id ricetta assente dall'array **favourites** dell'utente, viene aggiunto
- DELETE: Se id ricetta presente nell'array, viene rimosso



Toggle per Note Personali:

- ADD: Quando forniti **recipeId** e **text**, crea nuova nota con generazione ID univoco
- DELETE: Quando fornito **noteId**, rimuove nota specifica dall'array utente

3.3.4 Gestione Stati UI Centralizzata

Il modulo `ui.js` centralizza la gestione degli stati UI dinamici attraverso funzioni dedicate che garantiscono sincronizzazione bidirezionale tra dati persistenti e rappresentazione visuale. Funzioni chiave includono `favBtnDisplay(btn, recipeId)` per aggiornare icone preferiti e `revBtnDisplay(btn, recipeId)` per gestire testi pulsanti e visibilità modali. L'implementazione sfrutta `data attributes` per identificazione contestuale, `event delegation` per efficienza su contenuti dinamici, e `graceful degradation` per robustezza, assicurando aggiornamenti automatici e immediati post-interazione. Inoltre, il modulo sfrutta classi Bootstrap dedicate per la gestione degli stati UI: `overlay CSS` per indicare stati di loading durante operazioni asincrone e per disabilitare l'interfaccia durante operazioni critiche, classi `is-valid` e `is-invalid` per fornire feedback real-time durante la validazione dei form, e una navbar dinamica che si aggiorna automaticamente in base allo stato di autenticazione dell'utente. Quest'ultima mostra menu contestuali differenziati: per utenti loggati include un dropdown con username, impostazioni e logout, mentre per guest limita l'accesso a ricerca e login.

4. TEST e COMPATIBILITA'

4.1 Requisiti di Compatibilità

- **ES6 Modules:** Supporto nativo per import/export di moduli JavaScript
- **Web Crypto API:** Disponibile solo su HTTPS su mobile
- **Web Storage API:** `localStorage/sessionStorage` abilitati
- **Connessione Internet:** Richiesta per caricamento ricette da TheMealDB API

4.2 Test Funzionali

L'applicazione è stata sottoposta a test funzionali completi per verificare il corretto funzionamento di tutte le funzionalità principali:

- **Workflow Utente Completo:** Registrazione, autenticazione, navigazione, gestione preferiti e recensioni
- **Gestione Errori:** Validazione input, gestione casi limite, `graceful degradation`
- **Persistenza Dati:** Salvataggio e recupero dati attraverso sessioni browser
- **Integrazione API:** Comunicazione con TheMealDB e gestione cache

I browser utilizzati per test sono:

- Desktop: Edge 141, Chrome 141
- Mobile: Chrome 141, Brave 1.83.109 (Chromium 141) su Android 15

4.3 Test Responsive Design

Il design responsive è stato testato su diverse dimensioni schermo e su dispositivi mobili per garantire un'esperienza utente ottimale su tutti i dispositivi.

5. CONCLUSIONI

5.1 Analisi Decisioni Architetture

Three-Layer Architecture:

- *Vantaggi:* Separazione concerns, modularità, manutenibilità del codice
- *Svantaggi:* Overhead per progetti semplici, complessità iniziale setup
- *Trade-off:* Scalabilità futura vs. semplicità immediata

Session Service Pattern:

- *Vantaggi:* API unificata, single source of truth, consistenza, debugging semplificato
- *Svantaggi:* Single point of failure, accentrimento responsabilità
- *Trade-off:* Controllo centralizzato vs. distribuzione responsabilità

Event Delegation Pattern:

- *Vantaggi:* Performance con contenuto dinamico, memoria ottimizzata, gestione centralizzata
- *Svantaggi:* Debugging più complesso, controllo granulare limitato
- *Trade-off:* Efficienza vs. granularità controllo eventi

5.2 Conformità Requisiti

Il progetto è stato sviluppato rispettando rigorosamente tutti i requisiti specificati nella consegna, implementando le seguenti caratteristiche:

Architettura Frontend-Only:

- *Implementazione:* Applicazione web sviluppata esclusivamente con HTML5, CSS3 e JavaScript ES6+ senza alcun backend server-side
- *Implicazioni:* Validazione dati esclusivamente client-side, sicurezza basata su hashing client-side, nessuna elaborazione server

Web Storage API per Persistenza Dati:

- *Implementazione:* Utilizzo esclusivo di localStorage/sessionStorage per memorizzazione di utenti, ricette, recensioni e preferenze in formato JSON
- *Implicazioni:* Persistenza locale limitata (~5MB), volatilità dati alla pulizia browser, isolamento per utente

Integrazione API TheMealDB:

- *Implementazione:* Caricamento iniziale completo del database ricette tramite API REST di TheMealDB con strategia di memorizzazione locale
- *Implicazioni:* Dipendenza da connessione internet per primo caricamento e immagini, fallback su dati locali per accessi successivi

Separazione Struttura e Presentazione:

- *Implementazione:* HTML5 puro per struttura semantica, CSS3 per presentazione responsive, JavaScript per interattività
- *Implicazioni:* Manutenibilità migliorata, accessibilità nativa, performance ottimizzata

Quattro Macro-Scenari Funzionali:

- *Implementazione:* Gestione profilo utente (registrazione/login/modifica/rimozione), ricerca ricette (testuale/categorica), ricettario personale (aggiunta/rimozione con note private), sistema recensioni (rating duale gusto/difficoltà)
- *Implicazioni:* Completezza funzionale secondo specifiche, UX coerente, integrità referenziale per dati utente

Formato Dati JSON:

- *Implementazione:* Tutti i dati serializzati in JSON per web storage, deserializzazione automatica per operazioni CRUD
- *Implicazioni:* Interoperabilità nativa JavaScript, parsing efficiente, storage compatto

5.3 Valutazione Scelte Tecnologiche

JavaScript ES6+ Modules:

- *Vantaggi:* Import/export nativi, sviluppo modulare
- *Svantaggi:* Compatibilità browser limitata
- *Trade off:* Codice più pulito ma deployment meno ottimizzato

Bootstrap 5:

- *Vantaggi:* UI consistency, componenti ready-to-use, responsive integrato
- *Svantaggi:* Bundle size, personalizzazione limitata, dipendenza esterna
- *Trade off:* Velocità sviluppo vs. controllo granulare

5.4 Strategia Progettuale

Semplicità di Sviluppo prima di Performance

Considerato il contesto del progetto e la **mole di dati limitata** (dataset utenti locale, ricette API esterna con cache), si è deliberatamente privilegiata la **semplicità di sviluppo** rispetto alle ottimizzazioni per performance avanzate, privilegiando **architettura chiara, codice facilmente manutenibile e tempi di sviluppo ottimizzati**.

Esempi di Scelte Semplificate:

- **Funzioni di interrogazione iterative:** Ricerche lineari senza caching interno per evitare complessità gestionale
- **Calcoli aggregati real-time:** Statistiche recensioni calcolate on-demand senza pre-computazione
- **Validazioni ridondanti:** Controlli duplicati tra layer per robustezza invece di ottimizzazione
- **Operazioni atomiche ripetute:** Privilegio per atomicità garantita con ripetizione operazioni

Razionale della Decisione:

- *Dataset piccolo:* Utenti limitati, ricette disponibili localmente, operazioni $O(n)$ accettabili
- *Manutenibilità:* Codice lineare e leggibile, debugging semplificato
- *Coerenza garantita:* Operazioni atomiche prevengono stati inconsistenti
- *Time-to-market:* Focus su completezza funzionale rispetto a ottimizzazione

5.5 Implicazioni per Ambiente di Produzione

Per un utilizzo in produzione con mole di dati significativa (migliaia di utenti, database estesi), sarebbero necessarie ottimizzazioni quali:

- **Sistema di caching:** Per ridurre calcoli ripetitivi
- **Database dedicato:** Per gestire volumi maggiori e concorrenza
- **Ottimizzazioni algoritmi:** Per migliorare performance con dataset grandi

5.6 Impatto sul Prodotto Finale

L'architettura implementata risulta pensata per il contesto accademico, con lo scopo di fornire equilibrio tra complessità implementativa e funzionalità complete. Le scelte tecnologiche sono state orientate verso conformità ai vincoli e tentativo di predisposizione per scalabilità futura.

Il sistema, pur operando nei limiti dei vincoli frontend-only, rappresenta un tentativo di architettura predisposta per integrazione backend con struttura estendibile che potrebbe facilitare evoluzioni verso soluzioni production-ready.